DOCUMENT DE CONCEPTION

Application de Gestion de Planning
Pour le secteur de la Restauration

Projet universitaire — 3eme année

Matière : Développement d'Application de Gestion

Année universitaire 2025 - 2026

Sommaire

Section	Titre	Page
1	Architecture générale de l'application	3
2	Description des modules	4
3	Détail des fonctions par module	5
4	Schéma de la base de données	9
5	Diagramme de navigation (flux utilisateur)	12
6	Récapitulatif des routes Flask	13

1. Architecture générale de l'application

1.1 Modèle architectural

L'application suit le patron de conception MVC (Modèle - Vue - Contrôleur), adapte au framework Flask :

Couche	Role	Technologies
Modèle (Model)	Gestion des données et interactions avec la base de données	Python, SQLite, SQLAlchemy
Vue (View)	Affichage de l'interface utilisateur dans le navigateur	HTML, CSS, JavaScript, Jinja2
Contrôleur (Controller)	Logique métier, traitement des requêtes HTTP	Python, Flask (routes/blueprints)

1.2 Structure des dossiers du projet

Organisation des fichiers du projet :

```
planning_restauration/
  app/                          # Dossier principal de l'application
    __init__.py                 # Initialisation Flask, config BDD
    models.py                   # Modeles SQLAlchemy (tables BDD)
    auth/                       # Module authentification
      __init__.py
      routes.py                # Routes : /login, /logout
      forms.py                 # Formulaire de connexion
    employees/                  # Module employes
      __init__.py
      routes.py                # Routes : /employees/*
      forms.py                 # Formulaires employe
    shifts/                     # Module creneaux
      __init__.py
      routes.py                # Routes : /shifts/*
      forms.py                 # Formulaire creneau
    absences/                   # Module absences/conges
      __init__.py
      routes.py                # Routes : /absences/*
      forms.py                 # Formulaire absence
    planning/                   # Module planning
      __init__.py
      routes.py                # Routes : /planning/*
    templates/                  # Fichiers HTML (Jinja2)
      base.html                # Template parent commun
      auth/login.html
      employees/               # Vues employes
      shifts/                  # Vues creneaux
      absences/                # Vues conges
      planning/                # Vues calendrier
    static/                     # Fichiers CSS, JS, images
```

```
    css/style.css
    js/planning.js          # Logique du calendrier
config.py                    # Configuration (BDD, cle secrete...)
run.py                       # Point d'entree de l'application
planning.db                  # Base de donnees SQLite
```

2. Description des modules

Module	Fichier principal	Responsabilité	Accessible par
auth	auth/routes.py	Connexion, déconnexion, gestion de session	Tous
Employées	employees/routes.py	CRUD des employés, liste, recherche	Manager uniquement
Shifts	shifts/routes.py	Création et gestion des créneaux horaires	Manager uniquement
Absences	absences/routes.py	Demandes de conges, approbation, absences	Manager + Employé
Planning	planning/routes.py	Affichage du calendrier semaine/mois	Manager + Employé
Model	models.py	Définition des tables et relations BDD	Interne (backend)

Chaque module est enregistré comme un Blueprint Flask, ce qui permet de l'isoler et de le tester indépendamment. Le fichier app/__init__.py se charge d'assembler tous les modules au démarrage.

3. Détail des fonctions par module

3.1 Module : auth

Fonction	Route HTTP	Methode	Description
login()	/login	GET / POST	Affiche le formulaire de connexion. Vérifie les credentials, crée la session et redirige selon le rôle.
logout()	/logout	GET	Détruit la session active et redirige vers la page de connexion.
load_user(id)	—	—	Fonction Flask-Login : charge l'utilisateur depuis la BDD à partir de son ID de session.

3.2 Module : employees

Fonction	Route HTTP	Methode	Description
list_employees()	/employees/	GET	Récupère la liste de tous les employés actifs. Supporte le filtrage par poste via paramètres URL.
create_employee()	/employees/new	GET / POST	Affiche le formulaire de création. Valide les données, hache le mot de passe, insère en BDD.
edit_employee(id)	/employees/<id>/edit	GET / POST	Charge l'employé par son ID, affiche le formulaire prérempli, enregistre les modifications.
toggle_employee(id)	/employees/<id>/toggle	POST	Bascule le statut actif/inactif de l'employé sans le supprimer.
view_employee(id)	/employees/<id>	GET	Affiche la fiche complète d'un employé avec ses prochains créneaux.

3.3 Module : shifts

Fonction	Route HTTP	Méthode	Description
create_shift()	/shifts/new	GET / POST	Formulaire de création d'un créneau. Vérifie les conflits d'horaires avant insertion.

Fonction	Route HTTP	Méthode	Description
edit_shift(id)	/shifts/<id>/edit	GET / POST	Modification d'un créneau existant. Revérifie les conflits.
delete_shift(id)	/shifts/<id>/delete	POST	Suppression d'un créneau après confirmation.
check_conflict()	/shifts/check-conflict	POST (AJAX)	Vérifie en temps réel si un créneau est en conflit avec un existant pour un employé.
duplicate_shift(id)	/shifts/<id>/duplicate	POST	Duplique un créneau sur une plage de dates sélectionnée.

3.4 Module : absences

Fonction	Route HTTP	Methode	Description
my_absences()	/absences/my	GET	Employé : liste de ses propres demandes de conges avec leur statut.
request_absence()	/absences/request	GET / POST	Employé : soumission d'une nouvelle demande de conge (dates + motif).
cancel_absence(id)	/absences/<id>/cancel	POST	Employé : annulation d'une demande dont le statut est encore 'En attente'.
list_absences()	/absences/	GET	Manager : liste de toutes les demandes, filtrables par statut ou employé.
approve_absence(id)	/absences/<id>/approve	POST	Manager : approuve une demande de conge.
reject_absence(id)	/absences/<id>/reject	POST	Manager : refuse une demande, avec commentaire optionnel.
add_absence()	/absences/add	GET / POST	Manager : saisie directe d'une absence (maladie, injustifiée) sans demande préalable.

3.5 Module : planning

Fonction	Route HTTP	Methode	Description
week_view()	/planning/week	GET	Affiche le planning de la semaine. Accepte un

Fonction	Route HTTP	Methode	Description
			paramètre ?date=YYYY-MM-DD pour naviguer.
month_view()	/planning/month	GET	Affiche la vue mensuelle avec résume des jours travaillés par employé.
get_planning_data()	/planning/data	GET (AJAX)	Retourne les données du planning en JSON pour affichage dynamique cote client.

3.6 Module : models

Classe/Modele	Table BDD	Description
User	users	Représente un utilisateur (employé ou manager). Contient les credentials et informations personnelles.
Post	posts	Représente un poste en restauration (serveur, cuisinier, etc.).
Shift	shifts	Représente un créneau de travail affecte à un employé.
Absence	absences	Représente une demande de conge ou une absence enregistrée.

4. Schema de la base de donnees

4.1 Table : users

Stocke tous les utilisateurs de l'application (employes et managers).

Colonne	Type SQL	Contraintes	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Identifiant unique auto-incrémente
nom	VARCHAR(50)	NOT NULL	Nom de famille de l'employé
prenom	VARCHAR(50)	NOT NULL	Prénom de l'employé
email	VARCHAR(120)	NOT NULL, UNIQUE	Email = identifiant de connexion
password_hash	VARCHAR(255)	NOT NULL	Mot de passe hache avec bcrypt
telephone	VARCHAR(20)	NULLABLE	Numéro de téléphone
post_id	INTEGER	FOREIGN KEY (posts.id)	Poste occupe dans l'établissement
contrat	VARCHAR(20)	NOT NULL	Type : CDI, CDD, Extra, Apprenti
date_embauche	DATE	NOT NULL	Date d'entrée dans l'établissement
role	VARCHAR(10)	NOT NULL, DEFAULT 'employe'	Valeur : 'manager' ou 'employé'
actif	BOOLEAN	NOT NULL, DEFAULT TRUE	Compte actif (TRUE) ou désactive (FALSE)
created_at	DATETIME	NOT NULL, DEFAULT NOW()	Date de création du compte

4.2 Table : posts

Referentiel des postes disponibles dans l'etablissement.

Colonne	Type SQL	Contraintes	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Identifiant unique
nom	VARCHAR(50)	NOT NULL, UNIQUE	Nom du poste (ex: Serveur, Cuisinier)
couleur	VARCHAR(7)	NOT NULL	Code couleur HEX pour le planning (ex: #FF5722)

Colonne	Type SQL	Contraintes	Description
description	TEXT	NULLABLE	Description optionnelle du poste

Postes pre-configures au demarrage : Serveur, Cuisinier, Hote/Hotesse, Bar, Caisse.

4.3 Table : shifts

Stocke tous les creneaux de travail affectes aux employes.

Colonne	Type SQL	Contraintes	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Identifiant unique du creneau
user_id	INTEGER	NOT NULL, FOREIGN KEY (users.id)	Employe affecte a ce creneau
post_id	INTEGER	NOT NULL, FOREIGN KEY (posts.id)	Poste occupe lors de ce service
date_service	DATE	NOT NULL	Jour du service
heure_debut	TIME	NOT NULL	Heure de debut du service
heure_fin	TIME	NOT NULL	Heure de fin du service
note	TEXT	NULLABLE	Commentaire du manager (optionnel)
created_at	DATETIME	NOT NULL, DEFAULT NOW()	Date de creation du creneau
created_by	INTEGER	FOREIGN KEY (users.id)	Manager ayant cree ce creneau

Contrainte metier : deux creneaux d'un meme employe ne peuvent pas se chevaucher sur la meme journee. Verifie cote serveur dans la fonction check_conflict().

4.4 Table : absences

Stocke les demandes de conges et les absences enregistrees.

Colonne	Type SQL	Contraintes	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Identifiant unique
user_id	INTEGER	NOT NULL, FOREIGN KEY (users.id)	Employe concerne par l'absence
type_absence	VARCHAR(30)	NOT NULL	Valeur : 'conge_paye',

Colonne	Type SQL	Contraintes	Description
			'sans_solde', 'maladie', 'injustifie'
date_debut	DATE	NOT NULL	Premier jour d'absence
date_fin	DATE	NOT NULL	Dernier jour d'absence
motif	TEXT	NULLABLE	Motif saisi par l'employe
statut	VARCHAR(15)	NOT NULL, DEFAULT 'en_attente'	Valeur : 'en_attente', 'approuve', 'refuse'
commentaire_manager	TEXT	NULLABLE	Commentaire du manager lors du traitement
traite_par	INTEGER	NULLABLE, FOREIGN KEY (users.id)	Manager ayant traite la demande
created_at	DATETIME	NOT NULL, DEFAULT NOW()	Date de soumission de la demande

4.5 Relations entre les tables

Table source	Colonne	Table cible	Type de relation
users	post_id	posts	Plusieurs employes -> 1 poste (N:1)
shifts	user_id	users	Plusieurs creneaux -> 1 employe (N:1)
shifts	post_id	posts	Plusieurs creneaux -> 1 poste (N:1)
shifts	created_by	users	Plusieurs creneaux -> 1 manager (N:1)
absences	user_id	users	Plusieurs absences -> 1 employe (N:1)
absences	traite_par	users	Plusieurs absences -> 1 manager (N:1)

5. Diagramme de navigation (flux utilisateur)

5.1 Flux Manager

Etape	Action	Page/Vue
1	Se connecter avec email + mot de passe	/login
2	Acceder au tableau de bord manager	/dashboard
3a	Gerer les employes (creer, modifier, desactiver)	/employees/
3b	Creer ou modifier des creneaux horaires	/shifts/new ou /shifts/<id>/edit
3c	Consulter et traiter les demandes de conges	/absences/
3d	Ajouter une absence manuelle	/absences/add
4	Visualiser le planning semaine ou mois	/planning/week ou /planning/month
5	Se deconnecter	/logout

5.2 Flux Employe

Etape	Action	Page/Vue
1	Se connecter avec email + mot de passe	/login
2	Acceder au tableau de bord employe	/dashboard
3a	Consulter son planning de la semaine	/planning/week
3b	Consulter son planning du mois	/planning/month
3c	Soumettre une demande de conge	/absences/request
3d	Consulter le statut de ses demandes	/absences/my
3e	Annuler une demande en attente	/absences/<id>/cancel
4	Se deconnecter	/logout

5.3 Regles de protection des acces

Regle	Description
Authentification requise	Toutes les pages (sauf /login) necessitent une session active. Sinon : redirection vers /login.
Role Manager	Les routes /employees/*, /shifts/* (creation/edition), /absences/ (liste manager) verifient que role == 'manager'.
Donnees personnelles	Un employe ne peut consulter que ses propres absences et son propre planning. Pas d'acces aux donnees des autres.

Regle	Description
Protection CSRF	Tous les formulaires POST incluent un token CSRF (Flask-WTF) pour eviter les attaques.

6. Recapitulatif des routes Flask

Route	Methode(s)	Fonction	Acces
/login	GET, POST	auth.login()	Public
/logout	GET	auth.logout()	Connecte
/dashboard	GET	main.dashboard()	Connecte
/employees/	GET	employees.list_employees()	Manager
/employees/new	GET, POST	employees.create_employee()	Manager
/employees/<id>	GET	employees.view_employee()	Manager
/employees/<id>/edit	GET, POST	employees.edit_employee()	Manager
/employees/<id>/toggle	POST	employees.toggle_employee()	Manager
/shifts/new	GET, POST	shifts.create_shift()	Manager
/shifts/<id>/edit	GET, POST	shifts.edit_shift()	Manager
/shifts/<id>/delete	POST	shifts.delete_shift()	Manager
/shifts/<id>/duplicate	POST	shifts.duplicate_shift()	Manager
/shifts/check-conflict	POST	shifts.check_conflict()	Manager
/absences/	GET	absences.list_absences()	Manager
/absences/my	GET	absences.my_absences()	Employe
/absences/request	GET, POST	absences.request_absence()	Employe
/absences/<id>/cancel	POST	absences.cancel_absence()	Employe
/absences/<id>/approve	POST	absences.approve_absence()	Manager
/absences/<id>/reject	POST	absences.reject_absence()	Manager
/absences/add	GET, POST	absences.add_absence()	Manager
/planning/week	GET	planning.week_view()	Connecte
/planning/month	GET	planning.month_view()	Connecte
/planning/data	GET	planning.get_planning_data()	Connecte

— Fin du Document de Conception —